

Программирование на С

Лекция 8

Типы данных

Целочисленные типы данных

Целочисленные типы предназначены для хранения **целых чисел**.

Они отличаются

- объёмом занимаемой памяти;
- диапазоном допустимых значений.

Тип данных

Тип данных определяет

- множество допустимых значений;
- множество допустимых операций над этими значениями.

Для языка **C** тип также определяет объём памяти, необходимый для хранения значения.

Основные целочисленные типы

Тип	Размер	Битов	Возможных значений	Диапазон
char	1 байт	8	$2^8 = 256$	$-2^7 \dots 2^7 - 1$
short	2 байта	16	$2^{16} = 65\,536$	$-2^{15} \dots 2^{15} - 1$
int	4 байта	32	$2^{32} \approx 4,3$ млрд	$-2^{31} \dots 2^{31} - 1$
long	4	32	2^{32}	$-2^{31} \dots 2^{31} - 1$
	или 8 байт*	или 64	или 2^{64}	или $-2^{63} \dots 2^{63} - 1$
long long	8 байт	64	$2^{64} \approx 18$ квинтиллионов	$-2^{63} \dots 2^{63} - 1$

*Указанные размеры являются типичными для современных компьютеров.

Беззнаковые целочисленные типы

Тип	Размер	Битов	Возможных значений	Диапазон
<code>unsigned char</code>	1 байт	8	$2^8 = 256$	$0 \dots 2^8 - 1$
<code>unsigned short</code>	2 байта	16	$2^{16} = 65\,536$	$0 \dots 2^{16} - 1$
<code>unsigned int</code>	4 байта	32	$2^{32} \approx 4,3$ млрд	$0 \dots 2^{32} - 1$
<code>unsigned long</code>	4	32	2^{32}	$0 \dots 2^{32} - 1$
	или 8 байт*	или 64	или 2^{64}	или $0 \dots 2^{64} - 1$
<code>unsigned long long</code>	8 байт	64	$2^{64} \approx 18$ квинтиллионов	$0 \dots 2^{64} - 1$

Представление целых чисел

Беззнаковые числа (unsigned)

$$a_{n-1}a_{n-2} \dots a_1a_0 = \sum_{i=0}^{n-1} a_i \cdot 2^i$$

где

- $a_i \in \{0, 1\}$
- a_{n-1} — старший бит
- a_0 — младший бит

Знаковые числа (дополнительный код)

$$a_{n-1}a_{n-2} \dots a_1a_0 = -a_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} a_i \cdot 2^i$$

где

- старший бит имеет отрицательный вес;
- остальные биты имеют обычные веса.

Представление положительных целых чисел

`unsigned char` и `char`

- Размер: 1 байт (8 бит)
- Все биты используются для хранения числа.

Десятичное	Двоичное
0	00000000
1	00000001
5	00000101
42	00101010
127	01111111

Пока старший бит равен 0, число считается положительным.

Представление отрицательных чисел

unsigned char

Отрицательные значения хранить не может.

Диапазон: 0 ... 255

char

Используется **дополнительный код (Two's complement)**.

Значение	Биты
-1	11111111
-2	11111110
-5	11111011
-42	11010110
-128	10000000

Как выбирать тип?

Используйте тип, диапазона которого **достаточно** для вашей задачи.

Нет смысла хранить возраст человека в переменной типа `long long`.

Нет смысла хранить население Земли в переменной типа `char`.

Примеры

```
char grade = 5;
```

```
short year = 2026;
```

```
int population = 145000000;
```

```
long long stars = 2000000000000LL;
```

Что использовать на практике?

- `int` — основной целочисленный тип.
- `char` — для хранения символов.
- `short` используется редко.
- `long` и `long long` — когда диапазона `int` недостаточно.

Вещественные числа

Целые типы подходят только для целых значений.

Например:

- температура: 23.5 °C
- масса: 74.8 кг
- скорость: 12.75 м/с
- число π : 3.1415926...

Для хранения таких значений используются типы `float` и `double`.

Типы вещественных чисел

Тип	Размер	Примерная точность	Диапазон
float	4 байта	около 7 цифр	$\pm 3.4 \times 10^{38}$
double	8 байт	около 15–16 цифр	$\pm 1.8 \times 10^{308}$

`double` используется значительно чаще, чем `float`.

Почему нельзя хранить дробные числа как целые?

Число

123456789

легко записать в двоичном виде.

Но как хранить

12345.6789 или 0.000000000123 или 9876543210000000.0

Количество цифр до и после запятой постоянно меняется. Поэтому используется другое представление.

Научная запись числа

Любое число можно записать в виде

$$m \times 10^e$$

где $m \in [0, 10)$ (то есть целая часть имеет одну цифру в записи).

Обычная запись	Научная запись
1234	1.234×10^3
0.0056	5.6×10^{-3}
987000000	9.87×10^8

Компьютер использует точно такую же идею, но основание равно 2, а не 10.

Представление числа в памяти

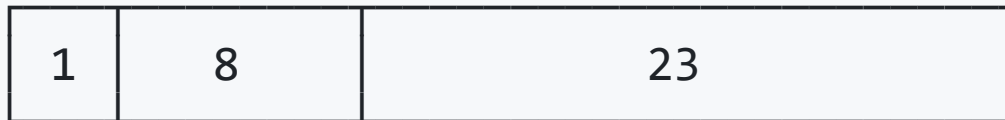
Каждое число состоит из трёх частей.

знак	порядок	мантисса
------	---------	----------

- знак определяет положительное или отрицательное число;
- порядок определяет степень двойки;
- мантисса хранит значащие цифры числа.

Формат IEEE 754

float (32 бита)



знак порядок

мантисса

double (64 бита)



знак порядок

мантисса

Значение числа

Для нормализованных чисел

$$(-1)^{sign} \times 1.mantissa \times 2^{exponent-bias}$$

где

- `sign` — знак;
- `mantissa` — дробная часть;
- `exponent` — порядок;
- `bias` — специальное смещение.

Почему возникают ошибки округления?

Некоторые десятичные дроби невозможно точно представить в двоичной системе.

Например `0.1` хранится только приблизительно.

Поэтому `0.1 + 0.2` равно `0.30000000000000004`

Вещественные числа почти всегда являются приближёнными.

Почему $0.1 + 0.2 \neq 0.3$

Значение	IEEE 754 (double)
0.1	0x3FB999999999999A
0.2	0x3FC999999999999A
0.3	0x3FD3333333333333
$0.1 + 0.2$	0x3FD3333333333334

Последний бит отличается.

Поэтому

```
0.1 + 0.2 == 0.3
```

имеет значение `false`.

Операции языка C

Группа	Операции
Арифметические	<code>+ - * / %</code>
Сравнения	<code>< <= > >= == !=</code>
Логические	<code>! && </code>
Присваивание	<code>=</code>
Арифметические с присваиванием	<code>+= -= *= /= %=</code>
Инкремент и декремент	<code>++ --</code>

Группа	Операции
Побитовые	<code>~ & ^ << >></code>
Побитовые с присваиванием	<code>&= = ^= <<= >>=</code>
Адресные	<code>& *</code>
Доступ к составным типам	<code>. -> []</code>
Вызов функции	<code>()</code>
Прочие	<code>?: sizeof ,</code>

Свойства операций

Каждая операция характеризуется несколькими свойствами.

- название;
- знак;
- приоритет;
- аргументность;
- ассоциативность.

Именно эти свойства определяют, как вычисляется выражение.

Приоритет

Приоритет определяет, какая операция выполняется раньше.

Например

$$2 + 3 * 4$$

Сначала выполняется умножение, затем сложение.

Арность

Арность — это количество операндов, с которыми работает операция.

$-a$

унарная операция

$a + b$

бинарная операция

$a ? b : c$

тернарная операция

Ассоциативность

Ассоциативность определяет порядок вычисления операций **одного приоритета**.

Например

$10 - 5 - 2$

вычисляется как

$(10 - 5) - 2$

потому что операция **-** имеет левую ассоциативность.

Арифметические операции

Название	Знак	Приоритет	Арность	Ассоциативность
Унарный плюс	+	высокий	унарная	правая
Унарный минус	-	высокий	унарная	правая
Умножение	*	средний	бинарная	левая
Деление	/	средний	бинарная	левая
Остаток от деления	%	средний	бинарная	левая
Сложение	+	низкий	бинарная	левая
Вычитание	-	низкий	бинарная	левая

Унарные операции

Унарная операция выполняется над одним операндом.

Операция	Назначение
+	возвращает значение операнда
-	возвращает противоположное значение

Результатом является **новое значение**.

Исходная переменная не изменяется.

Унарный плюс

```
int test = 17;  
int result = 0;  
  
result = +test;  
  
printf("%d\n%d\n", test, result);
```

`+test` возвращает значение переменной `test` .

```
17  
17
```

Унарный минус

```
int test = 17;  
int result = 0;  
  
result = -test;  
  
printf("%d\n%d\n", test, result);
```

`-test` возвращает число с противоположным знаком.

```
17  
-17
```

Значение переменной `test` при этом не изменяется.

Умножение

Целые числа

```
int a = 5, b = 6;  
printf("%d\n", a * b);
```

30

Вещественные числа

```
float a = 5.5f, b = 3.2f;  
printf("%f\n", a * b);
```

17.600000

Символы

```
char a = 'a', b = 'b';  
printf("%d\n", a * b);
```

9506

Перед выполнением операции символы автоматически преобразуются в целые числа.

'a' → 97
'b' → 98

Деление

Целые числа

```
int a = 30, b = 6;  
printf("%d\n", a / b);
```

5

```
int a = 5, b = 2;  
printf("%d\n", a / b);
```

2

Вещественные числа

```
float a = 5.5f, b = 3.2f;  
printf("%f\n", a / b);
```

1.718750

При делении целых чисел дробная часть отбрасывается.

Деление на ноль

Целые числа

```
int a = 30;  
int b = 0;  
printf("%d\n", a / b);
```

Деление целого числа на ноль приводит к **неопределённому поведению** (undefined behavior).

Результат заранее неизвестен.

Вещественные числа

```
float a = 30.0f;  
float b = 0.0f;  
printf("%f\n", a / b);
```

inf

```
float a = -30.0f;  
float b = 0.0f;  
printf("%f\n", a / b);
```

-inf

Стандарт IEEE 754 определяет специальные значения $+\infty$ и $-\infty$, поэтому деление вещественного числа на ноль не является неопределённым поведением.

Не число (NaN)

```
float a = 0.0f;  
float b = 0.0f;  
printf("%f\n", a / b);
```

nan

NaN (Not a Number) обозначает результат вычисления, который не является числом.

Например:

- `0.0 / 0.0`
- `sqrt(-1)`
- `∞ - ∞`

Явное приведение типов

Оператор приведения записывается в виде

```
(<type>)
```

Он вычисляет **новое значение** указанного типа.

При приведении вещественного числа к целому дробная часть **отбрасывается**.

Явное приведение типов

Если преобразование типов выполняется намеренно, его рекомендуется записывать явно.

Так код становится понятнее.

Без приведения

```
int a = 3, b = 4;  
printf("%.2f\n", a / b);
```

0.00

С приведением

```
int a = 3, b = 4;  
printf("%.2f\n", (float)a / b);
```

0.75

Приведение к целому типу

```
float x = 3.99f;  
printf("%d\n", (int)x);
```

3

При приведении к целому типу дробная часть отбрасывается.

Округление не выполняется.

Оператор %

Оператор % определён только для целых типов.

Он вычисляет остаток, связанный с результатом целочисленного деления.

Всегда выполняется равенство

$$a = (a/b) \times b + (a \% b)$$

Оператор %

Всегда выполняется равенство

$$a = (a/b) \times b + (a \% b)$$

```
int a = 30, b = 7;  
printf("%d\n", a % b);
```

2

```
int a = -30, b = 7;  
printf("%d\n", a % b);
```

-2

Знак результата совпадает со знаком первого операнда (`a`).

Логические операции

- Результатом операции является **истина** или **ложь**.
- Логические операции (`!` , `&&` , `||`) всегда возвращают `0` или `1` .
`0` — ложь. `1` — истина.
- При проверке условия любое значение, отличное от `0` , интерпретируется как истина.
- Чаще всего используются в условиях `if` , `while` , `for` .

Операция	Оператор	Приоритет	Арность	Особенности
Логическое НЕ	<code>!</code>	высокий	унарная	справа налево
Логическое И	<code>&&</code>	ниже <code>!</code>	бинарная	короткое замыкание
Логическое ИЛИ	<code> </code>	ниже <code>&&</code>	бинарная	короткое замыкание

Короткое замыкание

```
int a = 0;  
  
if (a != 0 && 10 / a > 1)  
{  
    printf("Hello\n");  
}
```

Сначала вычисляется `a != 0`

Она равна `0` (ложь), поэтому правая часть `10 / a > 1` не вычисляется.

Деления на ноль не происходит.

Таблица истинности

A	B	!A	A && B	A B
0	0	1	0	0
0	1	1	0	1
1	0	0	0	1
1	1	0	1	1

Здесь используются значения, которые возвращают логические операции:

0 — ложь, 1 — истина.

Логическое отрицание

```
#include <stdio.h>

int main() {
    float a = 5.6;
    int b = 34;

    printf("%d\n", !a);
    printf("%d\n", !(b - b));
    printf("%d\n", !((b - b) * a));
    printf("%f\n", !(b - b) * a);
}
```

```
0
1
1
5.600000
```

Логическое И (&&)

```
float a = 5.6, b = 3.4, c = 0;  
int d = 3, e = 7, f = 0;
```

```
(a - a) && (c = b);  
printf("%f\n", c);
```

```
(c = b) && (a - a);  
printf("%f\n", c);
```

```
(e * f) && (f = d);  
printf("%d\n", f);
```

```
(f = d) && (d = e * f);  
printf("%d\n", f);  
printf("%d\n", d);
```

Что будет выведено?

```
0.000000  
3.400000  
0  
3  
21
```

Если левый операнд равен 0, правый операнд не вычисляется.

Логическое ИЛИ (||)

```
float a = 5.6, b = 3.4, c = 0;  
int d = 3, e = 7, f = 0;
```

```
a || (c = b);  
printf("%f\n", c);
```

```
(c = b) || (a - a);  
printf("%f\n", c);
```

```
e || (f = d);  
printf("%d\n", f);
```

```
(f = d) || (d = e * f);  
printf("%d\n", f);  
printf("%d\n", d);
```

Что будет выведено?

```
0.000000  
3.400000  
0  
3  
3
```

Если левый операнд не равен 0, правый операнд не вычисляется.

Сложные логические выражения

```
float a = 5.6, b = 3.4, c = 0;  
int d = 3, e = 7, f = 0;
```

```
d || (f = e && a);  
printf("%d\n", f);
```

```
(f = e && a) || (c = b);  
printf("%d\n", f);  
printf("%f\n", c);
```

```
((f = e) && a) || (c = b);  
printf("%d\n", f);  
printf("%f\n", c);
```

```
(f && (a - a)) || (c = b);  
printf("%f\n", c);
```

Что будет выведено?

```
0  
1  
0.000000  
7  
0.000000  
3.400000
```

Операторы сравнения

- Сравнивают два значения.
- Результатом всегда является `0` или `1`.
- Возвращают только `0` или `1`.

Операция	Оператор	Приоритет	Арность
меньше	<code><</code>	средний	бинарная
меньше или равно	<code><=</code>	средний	бинарная
больше	<code>></code>	средний	бинарная
больше или равно	<code>>=</code>	средний	бинарная
равно	<code>==</code>	низкий	бинарная
не равно	<code>!=</code>	низкий	бинарная

Операторы сравнения

```
int a = 3, b = 5;

printf("%d\n", a < b);
printf("%d\n", a >= b);
printf("%d\n", a + 2 == b);
printf("%d\n", a != b);
printf("%d\n", a == b);
```

Что будет выведено?

1
0
1
1
0

Что выведет программа?

```
int x = 40;  
  
if (30 < x < 50)  
    puts("TRUE");  
else  
    puts("FALSE");
```

TRUE

```
int x = 40;  
  
if (45 < x < 60)  
    puts("TRUE");  
else  
    puts("FALSE");
```

TRUE

В языке **C** нельзя записывать двойное сравнение так, как в математике.

Почему так происходит?

```
30 < x < 50
```

вычисляется как

```
(30 < x) < 50
```

При `x = 40` :

```
30 < 40      // 1  
1 < 50       // 1
```

```
45 < x < 60
```

вычисляется как

```
(45 < x) < 60
```

При `x = 40` :

```
45 < 40    // 0  
0 < 60     // 1
```

Если нужно проверить диапазон, используйте логическое И:

```
30 < x && x < 50
```

Что выведет программа?

```
int x = -20;  
  
if (-30 < x < -10)  
    puts("TRUE");  
else  
    puts("FALSE");
```

FALSE

```
int x = -20;  
  
if (-10 < x < 1)  
    puts("TRUE");  
else  
    puts("FALSE");
```

TRUE

```
int x = 40;
```

```
if (30 < x && x < 50)  
    puts("TRUE");  
else  
    puts("FALSE");
```

TRUE

```
int x = 40;
```

```
if (45 < x && x < 60)  
    puts("TRUE");  
else  
    puts("FALSE");
```

FALSE

```
int x = -20;
```

```
if (-30 < x && x < -10)  
    puts("TRUE");  
else  
    puts("FALSE");
```

TRUE

```
int x = -20;
```

```
if (-10 < x && x < 1)  
    puts("TRUE");  
else  
    puts("FALSE");
```

FALSE

Операция равенства ==

```
const int TEST = 5;  
int a = 3;  
  
printf("%d\n", a == TEST);  
printf("%d\n", a = TEST);
```

```
0  
5
```

```
const int TEST = 5;  
int a = 3;  
  
printf("%d\n", TEST == a);  
printf("%d\n", TEST = a);
```

```
error: cannot assign to  
variable 'TEST'  
with const-qualified type  
'const int'
```

Операция равенства ==

Легко перепутать == и = .

Если сравнивается переменная с константой, рекомендуется писать **константу слева**:

```
TEST == a
```

Тогда случайная замена == на = приведёт к ошибке компиляции.

Побитовые операции

- Работают только с целыми числами.
- Выполняются над каждым битом независимо.
- Не изменяют свои операнды.
- Результатом является целое число.

Чтобы понять результат операции, обычно число переводят в двоичную систему счисления.

Побитовые операции

Операция	Оператор	Приоритет	Арность
Побитовое НЕ	~	высокий	унарная
Побитовое И	&	средний	бинарная
Побитовое исключающее ИЛИ	^	ниже &	бинарная
Побитовое ИЛИ		ниже ^	бинарная
Сдвиг влево	<<	средний	бинарная
Сдвиг вправо	>>	средний	бинарная

Побитовое И (&)

```
char a = 10;  
char b = 7;  
  
printf("%d\n", a & b);
```

2

Значение	Двоичный вид
a	00001010
b	00000111
a & b	00000010

Побитовое И часто используется для проверки отдельных битов с помощью маски.

Побитовое ИЛИ (|)

```
char a = 10;  
char b = 7;  
  
printf("%d\n", a | b);
```

15

Значение	Двоичный вид
a	00001010
b	00000111
a b	00001111

Побитовое ИЛИ часто используется для объединения нескольких флагов в одно значение.

Побитовое исключающее ИЛИ (^)

```
char a = 10;  
char b = 7;  
  
printf("%d\n", a ^ b);
```

13

Значение	Двоичный вид
a	00001010
b	00000111
a ^ b	00001101

Побитовое исключающее ИЛИ используется во многих алгоритмах обработки данных и шифрования.

Побитовое НЕ (~)

```
char a = 10;  
char b = -7;  
  
printf("%d\n", ~a);  
printf("%d\n", ~b);
```

-11
6

Значение	Двоичный вид
a	00001010
b	11111001
~a	11110101
~b	00000110

Для целых чисел в дополнительном коде:

$$\sim x == -x - 1$$

Побитовый сдвиг влево (<<)

```
int a = 17, b = 6;
```

```
printf("%d\n", a << b);  
printf("%d\n", b << a);  
printf("%d\n", a << 3);  
printf("%d\n", b << 3);
```

```
1088  
786432  
136  
48
```

Выражение	Результат
$17 \ll 6$	17×2^6
$6 \ll 17$	6×2^{17}
$17 \ll 3$	17×2^3
$6 \ll 3$	6×2^3

Сдвиг влево эквивалентен умножению числа на 2^n , где n — количество сдвигов. Освободившиеся справа биты заполняются нулями.

Побитовый сдвиг вправо (>>)

```
int a = 17, b = 6;

printf("%d\n", a >> b);
printf("%d\n", b >> a);
printf("%d\n", a >> 3);
printf("%d\n", b >> 3);
```

```
0
0
2
0
```

Выражение	Результат
<code>17 >> 6</code>	<code>17 / 2⁶</code>
<code>6 >> 17</code>	<code>6 / 2¹⁷</code>
<code>17 >> 3</code>	<code>17 / 2³</code>
<code>6 >> 3</code>	<code>6 / 2³</code>

Для неотрицательных чисел сдвиг вправо эквивалентен целочисленному делению на 2^n , где n — количество сдвигов. При сдвиге положительных чисел освободившиеся слева биты заполняются нулями. Для отрицательных чисел результат зависит от реализации и обычно сохраняет знак.

Операция присваивания

- Изменяет значение левого операнда.
- Возвращает присвоенное значение.

Операция	Оператор	Приоритет	Арность	Ассоциативность
присваивание	=	самый низкий	бинарная	справа налево

Операция присваивания

При цепочке присваиваний сначала выполняется самое правое:

```
a = b = c = 10;
```

ЭКВИВАЛЕНТНО

```
c = 10;  
b = c;  
a = b;
```

L-value и R-value

L-value	R-value
Обозначает существующий объект в памяти	Представляет значение
Обычно имеет адрес	Может не иметь адреса
Может быть изменён операцией присваивания	Используется при вычислении выражений

L-value и R-value

```
int a = 5;
```

```
a = 10;
```

```
a = a + 1;
```

- `a` слева — L-value
- `a` справа — L-value
- `10` — R-value
- `1` — R-value
- `a + 1` — R-value

L-value определяется не тем, где находится в выражении, а тем, что обозначает объект в памяти.

L-value и R-value (формально)

L-value — выражение, обозначающее объект.

R-value — значение, полученное в результате вычисления выражения.

```
int a = 5;  
int b = a + 1;
```

- `a` — L-value
- `a + 1` — R-value
- `5` и `1` — R-value

L-value и R-value — свойства выражений, а не переменных или типов данных.

Арифметические операции с присваиванием

Оператор	Эквивалент
<code>+=</code>	<code>a = a + b</code>
<code>-=</code>	<code>a = a - b</code>
<code>*=</code>	<code>a = a * b</code>
<code>/=</code>	<code>a = a / b</code>
<code>%=</code>	<code>a = a % b</code>

Свойство	Значение
Приоритет	самый низкий
Арность	бинарная
Ассоциативность	правая

Зачем нужны операции с присваиванием?

```
int myVeryVeryVeryVeryLongNameVariable = 5;  
int b = 7;
```

```
myVeryVeryVeryVeryLongNameVariable =  
    myVeryVeryVeryVeryLongNameVariable + b;
```

```
int myVeryVeryVeryVeryLongNameVariable = 5;  
int b = 7;
```

```
myVeryVeryVeryVeryLongNameVariable += b;
```

Операции с присваиванием делают код короче и избавляют от повторения длинных имен переменных.

Побитовые операции с присваиванием

Оператор	Эквивалент
<code>&=</code>	<code>a = a & b</code>
<code> =</code>	<code>a = a b</code>
<code>^=</code>	<code>a = a ^ b</code>
<code><<=</code>	<code>a = a << b</code>
<code>>>=</code>	<code>a = a >> b</code>

Свойство	Значение
Приоритет	самый низкий
Арность	бинарная
Ассоциативность	правая

Обмен значений с помощью XOR

```
int a = 10;
int b = 7;

printf("%d\t%d\n", a, b);

a ^= b;
b ^= a;
a ^= b;

printf("%d\t%d\n", a, b);
```

```
10      7
7       10
```

После операции	a	b
начальное состояние	10	7
a ^= b	a ^ b	7
b ^= a	a ^ b	10
a ^= b	7	10

Префиксный и постфиксный инкремент

Префиксная форма	Постфиксная форма
<code>++i</code>	<code>i++</code>
Сначала увеличивает значение	Сначала возвращает текущее значение
Затем возвращает новое значение	Затем увеличивает значение
Является L-value	Не является L-value

Префиксный и постфиксный инкремент

```
int i = 5;  
  
printf("%d\n", ++i);  
printf("%d\n", i);
```

6
6

```
int i = 5;  
  
printf("%d\n", i++);  
printf("%d\n", i);
```

5
6

Если результат операции не используется, то `++i;` и `i++;` эквивалентны.

Корректное использование инкремента и декремента

```
int i = 5;

printf("%d\n", ++i);
printf("%d\n", i);

printf("%d\n", i++);
printf("%d\n", i);
```

```
6
6
6
7
```

```
int a[5] = {10, 20, 30, 40, 50};
int i = 0;

printf("%d\n", a[i++]);
printf("%d\n", a[i]);
```

```
10
20
```

Корректное использование инкремента и декремента

Инкремент и декремент безопасно использовать, если в одном выражении переменная изменяется только один раз.

Неправильное использование инкремента

```
int i = 5;  
  
printf("%d\n", ++i + i++);
```

Операнды `+` могут вычисляться в любом порядке.

Изменения `i` не упорядочены относительно друг друга, поэтому поведение программы не определено стандартом `C`.

`++i`

- изменяет `i`;
- вычисляет новое значение `i`.

`i++`

- вычисляет прежнее значение `i`;
- изменяет `i`.

Условная операция **?:**

Название	Знак	Приоритет	Арность	Ассоциативность
условная операция	?:	низкий	тернарная	правая

условие **?** выражение_1 **:** выражение_2

Если условие истинно, вычисляется **выражение_1** .

Если условие ложно, вычисляется **выражение_2** .

Из двух выражений после **?** и **:** вычисляется только одно.

Поиск максимума двух чисел

```
int a = 15;  
int b = 8;  
  
printf("%d\n",  
       a > b ? a : b);
```

15

```
int a = 5;  
int b = 8;  
  
printf("%d\n",  
       a > b ? a : b);
```

8

`a > b ? a : b`

- если `a > b` истинно — результатом будет `a`;
- иначе результатом будет `b`.

Операция `sizeof`

Название	Знак	Приоритет	Арность	Ассоциативность
получение размера	<code>sizeof</code>	высокий	унарная	правая

Оператор `sizeof` вычисляет размер типа или объекта в байтах.

```
sizeof(int)  
sizeof(char)  
sizeof(double)
```

```
int a;  
char c;  
  
sizeof(a)  
sizeof(c)
```

Результат операции имеет тип `size_t`.

Размер массива

```
int a[10];  
  
printf("%zu\n", sizeof(a));  
printf("%zu\n", sizeof(a[0]));
```

40
4

```
sizeof(a) / sizeof(a[0])
```

10

Так часто вычисляют количество элементов массива.

Приоритет операций

Самый высокий

() [] . -> i++ i--
++i --i + - ! ~ * & sizeof
* / %
+ -
<< >>
< <= > >=
== !=
&
^
|
&&
||
?:
= += -= ...

Самый низкий

Приоритет операций

При одинаковом приоритете порядок вычисления определяется ассоциативностью операторов.

Если есть сомнения — используйте круглые скобки.